Linux网络--Reactor模式层06 OneThreadOneLoop与eventfd

> ⓘ 系列导航
>
> 上一篇：Linux网络--Reactor模式层05 Connection回调与事件分发
> 系列索引：Linux网络--IO多路转接与Reactor系列总览

- One Thread One Loop 与 eventfd
  - 1. 单 Reactor 还必须回答什么
  - 2. One Thread One Loop 到底约束什么
  - 3. 单 Reactor 与 Multi-Reactor
    - 3.1 单 Reactor
    - 3.2 Multi-Reactor
  - 4. 多进程方案
  - 5. 多线程方案一：管道分配
  - 6. eventfd：更轻量的事件通知
    - 6.1 基本接口
    - 6.2 默认模式
    - 6.3 信号量模式
    - 6.4 常用标志
  - 7. eventfd 唤醒 EventLoop 的流程
  - 8. 跨线程任务队列
  - 9. Multi-Reactor 中的连接分配
    - 9.1 主循环 accept，worker 读业务
    - 9.2 多个 worker 一起 accept
  - 10. 负载均衡与公平性
    - 10.1 连接数均衡
    - 10.2 活跃度均衡
    - 10.3 单连接公平性
  - 11. 管道与 eventfd 的选择
  - 12. 完整结构板书
    - 12.1 第一层：单 Reactor 基础
    - 12.2 第二层：连接从哪里进入
    - 12.3 第三层：worker 怎样被唤醒
    - 12.4 第四层：连接怎样回到自己的 Loop
  - 13. 多 Reactor 工程检查清单

- 14. 系列收束：整条技术路线
  - 14.1 select
  - 14.2 poll
  - 14.3 epoll
  - 14.4 LT 与 ET
  - 14.5 Reactor
  - 14.6 One Thread One Loop

One Thread One Loop 与 eventfd

1. 单 Reactor 还必须回答什么

第五篇的单 Reactor 可以管理许多连接，但所有事件都由一个线程分发，所有回调也在该线程内执行。连接数扩大后，会逐渐遇到：

- 单核 CPU 的处理能力成为上限；
- 业务解析和计算占用事件循环时间；
- 一个慢回调拖慢全部连接；
- 希望使用多核，又不希望多个线程同时操作同一个 socket。

最直接的扩展方式是给不同线程分配不同事件循环：

```
Thread 1 -> EventLoop 1 -> epoll 1 -> 一组连接
Thread 2 -> EventLoop 2 -> epoll 2 -> 另一组连接
Thread 3 -> EventLoop 3 -> epoll 3 -> 另一组连接
```

这就是 **One Thread One Loop，OTOL**。

2. One Thread One Loop 到底约束什么

它并不是“整个服务器只能有一个线程”，也不是“所有资源都不能共享”，核心约束是：

1. 每个线程运行一个独立 EventLoop；
2. 每个连接有唯一的归属 EventLoop；
3. 同一连接的读、写、关闭状态由该 EventLoop 串行推进；
4. 不同线程不会同时操作同一个 Connection；
5. 跨线程只传递任务、通知或连接所有权。

这样可以保留 epoll 的高效等待，同时避免为每个连接加一把 I/O 锁。

```
连接 A 只由 Loop 1 修改
连接 B 只由 Loop 2 修改
连接 C 只由 Loop 3 修改

Loop 1 不直接修改 B、C 的 socket 状态
```

3. 单 Reactor 与 Multi-Reactor

3.1 单 Reactor

```
一个线程
  -> 等待监听 fd 和所有连接
  -> accept
  -> recv / send
  -> 业务回调
```

优点：

- 结构简单；
- 没有连接跨线程；
- 调试容易；
- 小规模服务足够高效。

限制：

- 单线程受到一个 CPU 核心限制；
- 慢回调会阻塞全部连接；
- 连接规模和业务计算增长后成为瓶颈。

3.2 Multi-Reactor

```
Main EventLoop
  -> 监听 listen fd
  -> accept
  -> 把新 fd 交给某个 Worker EventLoop
```

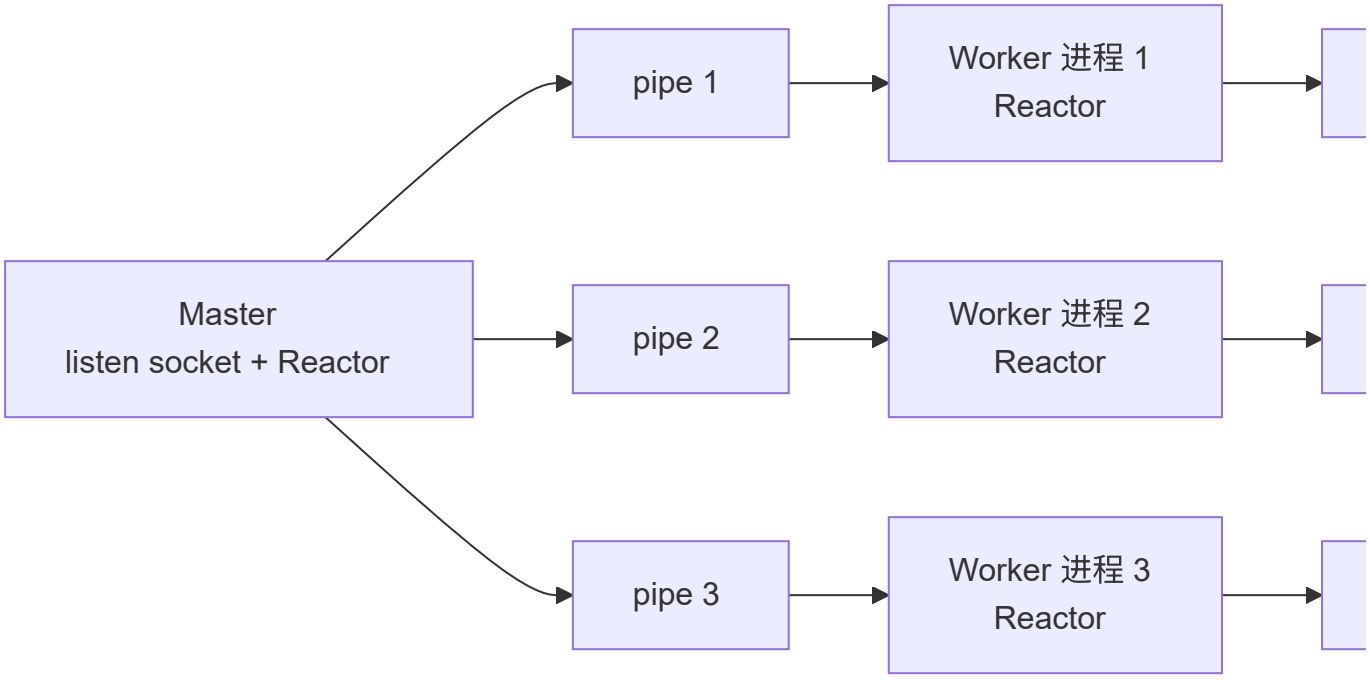
```
Worker EventLoop
  -> 管理自己的连接
  -> 独立 epoll_wait
  -> 独立读写和回调
```

关键不在“起了几个线程”，而在“fd 的所有权是否清晰”：

- 监听 socket 通常由主循环负责；
- 客户端 fd 只交给一个 worker；
- worker 在自己的线程里执行 `epoll_ctl ADD`；
- 同一个 fd 不会被两个 worker 同时读写；
- 连接关闭由所属 worker 完成。

4. 多进程方案

多进程也能利用同样的 Reactor 结构：



一种常见思路：

1. master 负责监听；
2. 通过管道把连接标识或分配信息交给 worker；
3. 每个 worker 维护自己的连接集合；
4. 每个 worker 处理自己接受的连接；
5. 任何连接都不在多个进程之间穿插 I/O。

多进程的边界比多线程更清晰，但进程间传递 fd、负载均衡、退出和错误恢复需要额外管理。

5. 多线程方案一：管道分配

主线程可以使用管道把新连接分配给工作线程。

```
Master 监听 listen fd
        |
        v
accept 得到 client fd
        |
        v
写入指定 worker 的 pipe
        |
        v
Worker 从 pipe 读取 4 字节 fd
        |
        v
把 fd 加入自己的 EventLoop
```

为什么管道也能用于线程间通信?

- 管道有读端和写端;
- 两端都是文件描述符;
- epoll 可以等待读端;
- 工作线程不必轮询共享内存标志。

最简单的 pipe 示例:

```
int pipefds[2];
pipe(pipefds);

std::thread worker([&] {
    char buffer[256];

    for (;;) {
        ssize_t n = read(pipefds[0], buffer, sizeof(buffer));
        if (n > 0) {
            std::cout << "worker received "
                      << std::string(buffer, n) << '\n';
```

```
        } else if (n == 0) {
            break;
        } else if (errno == EINTR) {
            continue;
        } else {
            break;
        }
    }
});

write(pipefds[1], "hello", 5);
worker.join();
```

实际传递 fd 时，不能只把整数当作业务消息随意复制到另一个进程。跨进程要使用 `SCM_RIGHTS` 等 Unix 域套接字机制；同一进程的线程共享文件描述符表，分配方式相对简单。实际服务还要明确由哪个线程执行 accept，避免多个线程同时 accept 造成连接分配失控。

6. eventfd：更轻量的事件通知

6.1 基本接口

```
#include <sys/eventfd.h>

int efd = eventfd(0, EFD_NONBLOCK | EFD_CLOEXEC);
```

eventfd 本身是一个文件描述符，可以被 epoll 监视：

```
epoll_event ev = {0};
ev.events = EPOLLIN;
ev.data.fd = efd;
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, efd, &ev);
```

当另一个线程向 eventfd 写入时，epoll 会把这个 `efd` 报告为可读，从而唤醒 EventLoop。

6.2 默认模式

eventfd 内部维护一个 64 位计数器。

```
write(1) -> counter += 1
write(2) -> counter += 2
read()   -> 返回累计值 3，并把 counter 清零
```

注意，默认模式的 `read` 不是简单减一，而是返回累计值并清空。如果连续发生三次通知，只需一次读取就能知道至少发生过通知。

6.3 信号量模式

创建时加上 `EFD_SEMAPHORE`：

```
int efd = eventfd(
    0,
    EFD_SEMAPHORE | EFD_NONBLOCK | EFD_CLOEXEC
);
```

此时每次 `read` 返回 `1`，并把计数器减一。适合“每一个通知都要单独消费一次”的场景。

6.4 常用标志

标志	作用
`EFD_NONBLOCK`	无事件时不阻塞，返回 `EAGAIN`
`EFD_CLOEXEC`	`exec` 时自动关闭，避免子进程继承
`EFD_SEMAPHORE`	使用每次减一的信号量语义

eventfd 传递的是“事件发生”这一事实，不负责承载业务消息。真正的任务应放在线程安全队列里。

7. eventfd 唤醒 EventLoop 的流程

```
Error parsing Mermaid diagram!

Parse error on line 9:
...(8 字节计数)    EFD-->>Loop: epoll_wait 返回
```

```
----------------------^
Expecting '+', '-', '()', 'ACTOR', got 'loop'
```

关键点：

1. 先放任务，再 `write(eventfd)`；
2. epoll 看到 eventfd 可读；
3. EventLoop 读取 eventfd，确认有通知；
4. 一次性取走队列中的全部任务；
5. 执行业务回调或连接迁移操作。

如果先写 eventfd 再放任务，消费者可能被唤醒后发现队列为空。相反顺序则安全得多。

8. 跨线程任务队列

一个简化设计：

```
class EventLoop {
public:
    void RunInLoop(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            tasks_.push(std::move(task));
        }

        uint64_t one = 1;
        write(event_fd_, &one, sizeof(one));
    }

private:
    void DrainWakeupFd() {
        uint64_t value;
        while (read(event_fd_, &value, sizeof(value)) > 0) {
        }

        std::queue<std::function<void()>> local;
        {
            std::lock_guard<std::mutex> lock(mutex_);
            std::swap(local, tasks_);
        }

        while (!local.empty()) {
```

```
            auto task = std::move(local.front());
            local.pop();
            task();
        }
    }

    int event_fd_;
    std::mutex mutex_;
    std::queue<std::function<void()>> tasks_;
};
```

`DrainWakeupFd` 只在 EventLoop 所属线程执行。任务本身也只在该线程中执行，因此连接 I/O 状态仍然保持单线程归属。

如果跨线程直接把 `Connection*` 交给 EventLoop 并在该线程执行回调，还需要保证对象在任务执行前没有被原线程释放。更稳妥的做法是使用所有权转移、引用计数或延迟回收。

9. Multi-Reactor 中的连接分配

9.1 主循环 accept，worker 读业务

```
Main Loop
    -> 监听 listen fd
    -> accept client fd
    -> 选择 worker
    -> 把 client fd 放入该 worker 的任务队列
    -> write(worker.eventfd)

Worker Loop
    -> 被 eventfd 唤醒
    -> 取出 client fd
    -> epoll_ctl ADD
    -> 处理连接读写
```

这是职责清晰的一种方案：主循环只负责接入，连接的事件处理全部在 worker。

9.2 多个 worker 一起 accept

也可以让多个 worker 共享监听 fd，再通过内核负载机制竞争 accept。实现时必须确认：

- 谁拥有监听 socket；
- 是否启用 `SO_REUSEPORT`；
- 是否使用 `EPOLLEXCLUSIVE`；
- accept 失败是否被错误地当作连接关闭；
- 某个 worker 是否因不公平分配而负载过重。

多个进程或线程共享同一个监听 fd 是可行的，但连接归属、退出流程和负载均衡要由应用明确管理。

10. 负载均衡与公平性

10.1 连接数均衡

最直接的分派方式：

```
worker = next_worker
next_worker = (next_worker + 1) % worker_count
```

它只保证新连接数量接近，不代表工作负载接近。有些连接每秒钟发送大量数据，有些连接长期空闲。

10.2 活跃度均衡

可参考：

- 当前连接数；
- 待发送字节数；
- 最近事件频率；
- 上次分配任务后 EventLoop 的队列长度。

不要让分配策略过度复杂，否则调度本身会成为新的锁竞争点。

10.3 单连接公平性

即使每个连接只属于一个 EventLoop，一个连接持续高速发送也可能长期占用该循环。可以在一次回调中限制：

- 最大读取字节数；
- 最大处理报文数；
- 最大发送字节数；
- 单连接单轮处理时间。

超过限制后需要重新登记事件或进入下一轮调度，不能让一个连接无限排空。

11. 管道与 eventfd 的选择

方式	特点	适合场景
pipe	可传递字节流，接口直观	简单唤醒、少量数据、跨进程通道
eventfd	固定 64 位计数器，开销较小	同级线程唤醒、任务队列通知
Unix domain socket	支持 `SCM_RIGHTS` 传 fd	跨进程传递文件描述符

eventfd 只告诉你有通知，不告诉你是谁；因此它通常与任务队列配合，而不是单独承载请求。

12. 完整结构板书

写事件默认是满足的，写事件不满足，sockfd托管给epoll，直到epoll检测到写事件满足

发送缓冲区有空间

发送缓冲区被写满

按需设置，大部分情况都是满足的，少量情况不满足

不能对写事件关心做常设，会导致epoller一直在就序，浪费CPU处理资源

默认写事件关心是没有开启的

如果我们主动开启epollout，无论发送条件是否满足，epollout事件默认会就绪一次！

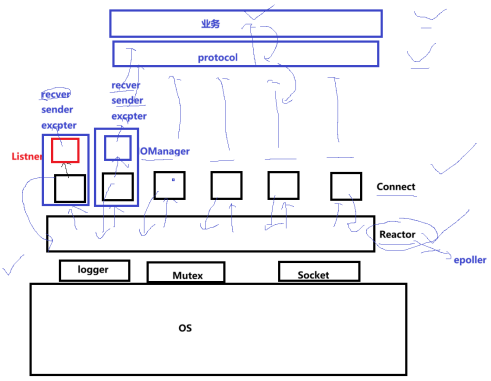
扩展：链接管理

扩展：多进程，多线程化

Reactor

扩展：智能指针的扩展

Reactor： 反应堆模式




```
uint16_t port = std::stoi(argv[1]);
// 1. 创建网络计算器
std::unique_ptr<Calculator> cal = std::make_unique<Calculator>();

// 2. 定义协议对象
std::unique_ptr<Protocol> protocol = std::make_unique<Protocol>(
    [&cal](Request & req)->Response{
        return cal->Execute(req);
    }
);

// 3. 创建listener对象 -> base connection
std::shared_ptr<Connection> listener = std::make_shared<Listener>([&protocol](std::string &inbuffer)->std::string{
    return protocol->ParseRequest(inbuffer);
}, port);

// 4. 创建一个Reactor
std::unique_ptr<Reactor> R = std::make_unique<Reactor>();

// 5. 把链接添加到TcpServer中
R->AddConnection(listener);

// 6. 事件派发逻辑
R->Loop();
```

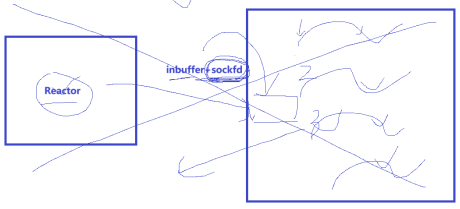


Reactor 模式是一种事件驱动的编程模式，用于高效地处理并发 I/O 操作。它通过一个或多个事件循环（Event Loop）来监听和处理各种事件（如网络请求、定时器事件等），从而实现高效的并发处理，而无需为每个连接创建一个线程或进程。

单Reactor有很多的使用价值 - Redis

多Reactor也有实用价值 —— Nginx

线程池内部



一个fd的真个生命周期，都应该在一个进程、线程内部全部处理完！

```
void Loop()
{
    int timeout = 2000;
    while (true)
    {
        // // 打印合法fd
        // DebugPrint();
        // 1. 事件派发
        LoopOnce(timeout);

        // 2. 链接管理
        // timeout = CheckExpiredLink();
    }
}
```

单进程

One Thread One Loop

OTOL

```
void Loop()
{
    int timeout = 2000;
    while (true)
    {
        // // 打印合法fd
        // DebugPrint();
        // 1. 事件派发
        LoopOnce(timeout);

        // 2. 链接管理
        // timeout = CheckExpiredLink();
    }
}
```

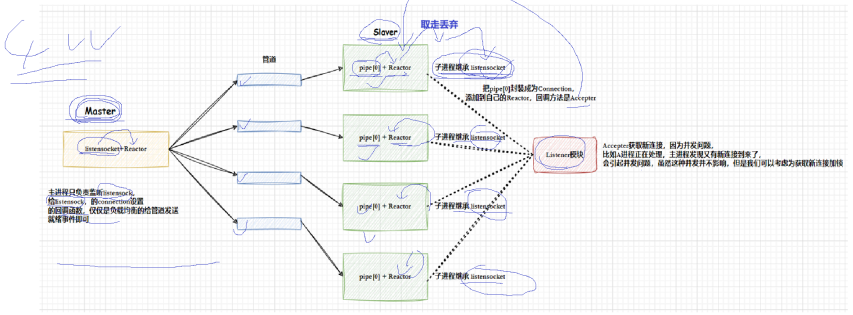
单进程

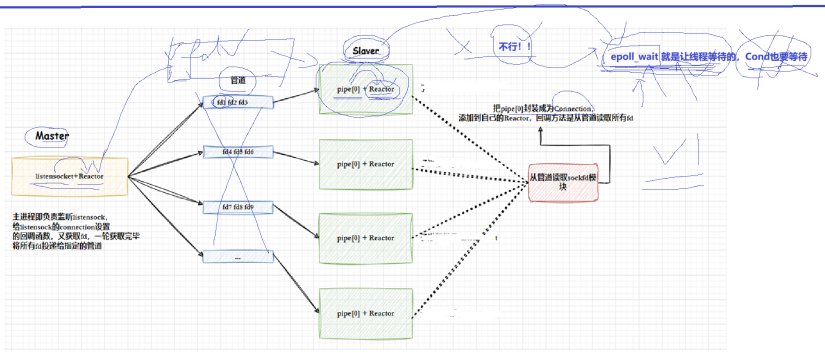
```
void Loop()
{
    int timeout = 2000;
    while (true)
    {
        // // 打印合法fd
        // DebugPrint();
        // 1. 事件派发
        LoopOnce(timeout);

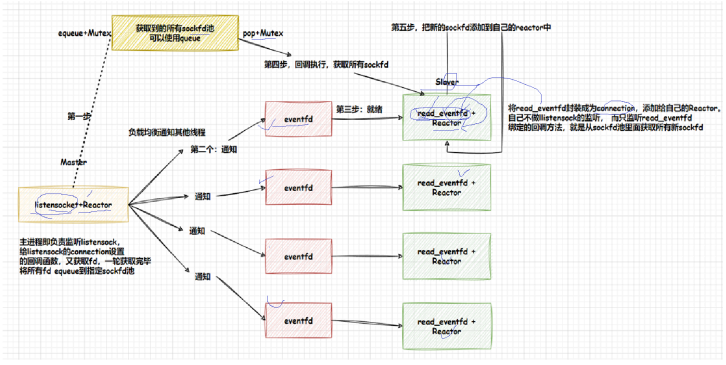
        // 2. 链接管理
```

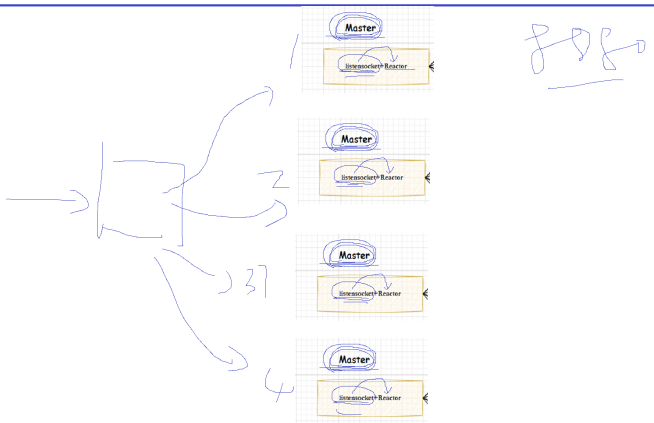
单进程

```
        // timeout = checkExpireDelta();
    }
}
```









这张板书把单 Reactor、master/worker、pipe、eventfd、连接建立和读写路径整合在同一张图中。建议分成四层阅读。

12.1 第一层：单 Reactor 基础

先确认已经有：

```
listen fd
epoll
Connection
Handler
```

单 Reactor 是后续扩展的基线。如果单 Reactor 的连接生命周期都不清楚，直接拆多线程只会把错误放大。

12.2 第二层：连接从哪里进入

master 监听 listen socket。新连接建立后，不能直接读业务数据，而是要把客户端 fd 的所有权交给一个 worker。

12.3 第三层：worker 怎样被唤醒

pipe 或 eventfd 是 worker 的“闹钟”。生产者放入任务后写一个通知，worker 的 epoll 因通知 fd 可读而返回。

12.4 第四层：连接怎样回到自己的 Loop

worker 从队列取出 fd，加入自己的 epoll。此后该连接的读、写、异常和关闭都由这个 worker 负责。

完整链路是：

```
listen fd 可读
    -> accept
```

```
    -> client fd
    -> 任务队列
    -> write(eventfd)
    -> worker epoll_wait 返回
    -> read(eventfd)
    -> 取出 client fd
    -> worker epoll_ctl ADD
    -> 后续 recv / send / close
```

13. 多 Reactor 工程检查清单

- 每个连接只有一个所属 EventLoop；
- 每个 fd 的 I/O 只由一个线程推进；
- eventfd 先排队任务，再写通知；
- EventLoop 读取 eventfd 后批量取任务；
- 任务队列有上限和关闭策略；
- 连接迁移有明确的所有权交接；
- 连接关闭时先从 epoll 移除；
- 当前回调结束前不释放仍可能被访问的对象；
- 业务重计算交给工作线程，不占用 EventLoop；
- 负载均衡指标不只看连接数；
- 一个连接不能无限占用单轮事件处理时间；
- 多线程退出时能唤醒所有 EventLoop 并安全清理任务。

14. 系列收束：整条技术路线

14.1 select

用位图表达 fd 集合，接口简单，但每轮都要重建、复制和扫描。

14.2 poll

用 `pollfd` 数组分离 `events` 与 `revents`，改善事件表达，但仍要提交整个数组并扫描结果。

14.3 epoll

用持久注册保存关注集合，通过回调把就绪项放入队列，等待时只取就绪子集。

14.4 LT 与 ET

LT 看当前条件，条件持续成立就持续通知；ET 看状态变化，要求应用在一次边沿中完成足够多的处理。

14.5 Reactor

用 EventLoop、Connection、缓冲区和回调把底层通知组织成可维护的服务端程序。

14.6 One Thread One Loop

让不同连接归属于不同事件循环，通过 pipe 或 eventfd 完成跨线程唤醒和连接迁移，在多核上扩展吞吐量。

最终可以把它压缩成一句话：

> epoll 负责发现“谁已经准备好”，Reactor 负责把这些事实组织成对象和回调，One Thread One Loop 负责让多个事件循环各自推进而不互相争抢连接。